Universidad de Guanajuato

Campus Irapuato-Salamanca



Licenciatura en Ingeniería de Datos e Inteligencia Artificial

Sistema de Gestión e Inventario Vehicular en C++

Programación Orientada a Objetos I

Cecil Quimey Azuara Yáñez

Salamanca, Guanajuato a 8 de octubre de 2026

Introducción

La empresa automotriz "Guanajuato Motors" requiere renovar su sistema de software para la administración de inventario y cálculo de precios de su flotilla vehicular. Debido a que las descripciones textuales de las marcas de los vehículos varían en tamaño, la información debe ser gestionada mediante la reserva dinámica de memoria en el *Heap*.

El sistema anterior presentaba fallos graves de estabilidad originados por una deficiente gestión de memoria, lo cual provocaba fugas de memoria (memory leaks) y conflictos por accesos cruzados debido al uso de copias superficiales (shallow copy). Para solucionar esta problemática de forma definitiva, se diseñó e implementó un nuevo núcleo en C++ fundamentado en la Regla de los Tres (constructor con asignación dinámica de memoria, constructor de copia profunda y operador de asignación), garantizando la integridad de los recursos y la eliminación de punteros colgantes (dangling pointers).

Esta actividad tiene por objetivo diseñar e implementar un programa estructurado en C++ para la gestión del inventario vehicular de "Guanajuato Motors", integrando los pilares fundamentales de la Programación Orientada a Objetos: Encapsulamiento (protección de atributos mediante getters y setters con validación), Gestión Dinámica de Memoria y Copia Profunda (deep copy en el Heap), Sobrecarga de Funciones (diferentes tipos de descuentos), Sobrecarga de Operadores (+, == y <<), así como Herencia y Polimorfismo mediante clases derivadas (Automovil y Motocicleta) que redefinen métodos para el cálculo de precios finales.

Código fuente

A continuación, se presenta la implementación completa en el lenguaje de programación C++ para “Guanajuato Motors”. El código incluye comentarios descriptivos que identifican la aplicación de cada uno de los pilares de POO requeridos en la práctica:

Código completo:

```cpp
#include <iostream>
#include <cstring>

using namespace std;

// Clase base "Vehiculo"
class Vehiculo{
private:
    char* marca;
    int year;
    double precio;

public:
    // Constructor por defecto
    Vehiculo() : year(0), precio(0.0){
        marca = new char[1]{'\0'}; // Memoria dinamica para "marca"
    }

    // Constructor parametrizado
    Vehiculo(const char* m, int y, double p) : year(y), precio(p > 0 ? p : 0.0){
        marca = new char[strlen(m) + 1]; // Memoria dinamica para "marca"
        strcpy(marca, m);
    }

    // Constructor de copia profunda (Deep Copy)
    Vehiculo(const Vehiculo& v) : year(v.year), precio(v.precio) {
        marca = new char[strlen(v.marca) + 1];
        strcpy(marca, v.marca);
        cout << "Copia hecha y memoria dinamica para \"marca\".\n" << endl;
    }

    // Operador de asignacion
    Vehiculo& operator=(const Vehiculo& v) {
        if (this != &v){
            delete[] marca;
            marca = new char[strlen(v.marca) + 1];
            strcpy(marca, v.marca);
            year = v.year;
            precio = v.precio;
        }
        return *this;
    }

    // Destructor
    virtual ~Vehiculo(){
        delete[] marca;
    }

    // Setters
    void setMarca(const char* m){
        delete[] marca;
        marca = new char[strlen(m) + 1];
        strcpy(marca, m);
    }

    void setYear(int y){
        year = y;
    }

    void setPrecio(double p){
        precio = (p > 0) ? p : precio;
    }

    // Getters
    const char* getMarca() const{
        return marca;
    }

    int getYear() const{
        return year;
    }

    double getPrecio() const{
        return precio;
    }

    // Sobrecarga de aplicarDescuento
    void aplicarDescuento(double porcentaje){
        precio -= (precio * (porcentaje / 100.0));
    }

    void aplicarDescuento(double monto, bool fijo){
        if (fijo && precio >= monto) precio -= monto;
    }

    // Calculo polimorfico base
    virtual double calcularPrecioFinal() const{
        return precio;
    }

    // Sobrecarga de operadores
    double operator+(const Vehiculo& v) const{
        return precio + v.precio;
    }

    bool operator==(const Vehiculo& v) const{
        return (year == v.year && precio == v.precio);
    }

    friend ostream& operator<<(ostream& os, const Vehiculo& v){
        return os << "Marca: " << v.marca << "\nYear: " << v.year << "\nPrecio Base: $" << v.precio;
    }
};
```

```
// Clase derivada "Automovil"
class Automovil : public Vehiculo{
private:
    int puertas;

public:
    Automovil() : Vehiculo(), puertas(4){}

    Automovil(const char* m, int y, double p, int numPuertas)
        : Vehiculo(m, y, p), puertas(numPuertas){}

    void setNumPuertas(int p){
        puertas = p;
    }

    int getNumPuertas() const{
        return puertas;
    }

    // 10%
    double calcularPrecioFinal() const{
        return getPrecio() * 1.10;
    }

};

// Clase derivada "Motocicleta"
class Motocicleta : public Vehiculo{
private:
    int cilindrada;

public:
    Motocicleta() : Vehiculo(), cilindrada(125){}

    Motocicleta(const char* m, int y, double p, int cc)
        : Vehiculo(m, y, p), cilindrada(cc){}

    void setCilindrada(int cc){
        cilindrada = cc;
    }

    int getCilindrada() const{
        return cilindrada;
    }

    // Recargo del 15% si la cilindrada >= 500 cc
    double calcularPrecioFinal() const{
        return (cilindrada >= 500) ? getPrecio() * 1.15 : getPrecio();
    }
};
```

```
// main
int main() {
    cout << "ESCENARIO 1\n" << endl;
    Automovil auto1("Ferrari", 2026, 6500000.00, 4);
    cout << auto1 << endl;

    Automovil auto2 = auto1; // Llama al constructor de copia
    cout << "\nauto2 (copia):\n" << auto2 << endl << endl;

    cout << "ESCENARIO 2\n" << endl;
    Motocicleta moto1("BMW", 2026, 750000.00, 725);

    Vehiculo* inventario[] = { &auto1, &moto1 };

    for (int i = 0; i < 2; i++) {
        cout << *inventario[i]
             << "\nPrecio Final: $" << inventario[i]->calcularPrecioFinal() << "\n" << endl;
    }

    cout << "ESCENARIO 3\n" << endl;
    cout << "Precio inicial auto1: $" << auto1.getPrecio() << endl;

    auto1.aplicarDescuento(10.0); // Descuento 10%
    cout << "Precio con el 10% de descuento: $" << auto1.getPrecio() << endl;

    auto1.aplicarDescuento(5000.0, true); // Descuento fijo $5000
    cout << "Precio con $5000 de descuento fijo: $" << auto1.getPrecio() << endl << endl;

    cout << "ESCENARIO 4\n" << endl;
    cout << "Suma de precios base (auto1 + moto1): $" << (auto1 + moto1) << endl;

    if (auto1 == auto2) {
        cout << "auto1 y auto2 tienen el mismo year y precio." << endl;
    } else {
        cout << "auto1 y auto2 NO tienen el mismo year y precio." << endl;
    }

    return 0;
}
```

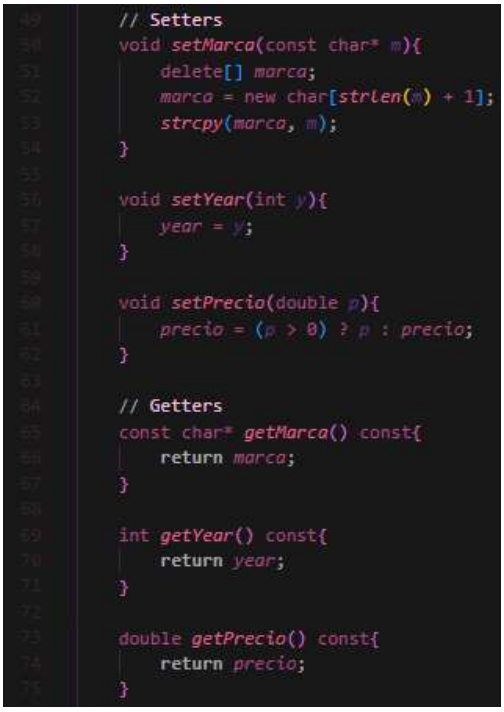
Memoria Dinámica y Copia Profunda: Se aplica mediante la reserva dinámica en el Heap (new char[]) para la variable marca en los constructores parametrizados y de copia. La duplicación byte a byte con la función strcpy, la cual asegura que cada objeto posea su propia dirección física de memoria independiente, evitando así punteros colgantes.

```
public:
    // Constructor por defecto
    Vehiculo() : year(0), precio(0.0){
        marca = new char[1]{'\0'}; // Memoria dinamica para "marca"
    }

    // Constructor parametrizado
    Vehiculo(const char* m, int y, double p) : year(y), precio(p > 0 ? p : 0.0){
        marca = new char[strlen(m) + 1]; // Memoria dinamica para "marca"
        strcpy(marca, m);
    }

    // Constructor de copia profunda (Deep Copy)
    Vehiculo(const Vehiculo& v) : year(v.year), precio(v.precio) {
        marca = new char[strlen(v.marca) + 1];
        strcpy(marca, v.marca);
        cout << "Copia hecha y memoria dinamica para \"marca\".\n" << endl;
    }
```

Encapsulación: Se protegen los miembros de datos (marca, year, precio) definiéndolos como privados (private), permitiendo el acceso controlado únicamente mediante getters y setters. En setPrecio se valida que el valor ingresado sea mayor a cero antes de asignarlo.



```
    // Setters
    void setMarca(const char* m){
        delete[] marca;
        marca = new char[strlen(m) + 1];
        strcpy(marca, m);
    }

    void setYear(int y){
        year = y;
    }

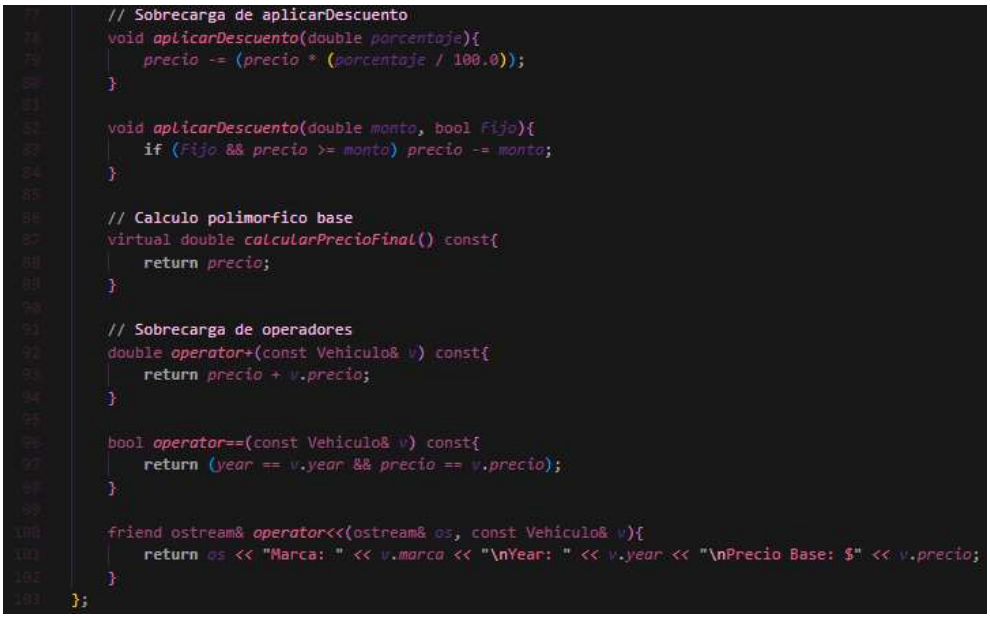
    void setPrecio(double p){
        precio = (p > 0) ? p : precio;
    }

    // Getters
    const char* getMarca() const{
        return marca;
    }

    int getYear() const{
        return year;
    }

    double getPrecio() const{
        return precio;
    }
```

Sobrecarga: Se demuestra en la sobrecarga de la función miembro aplicarDescuento (diferenciando por firma de parámetros: porcentaje double vs. monto con indicador bool) y en la sobrecarga de los operadores aritméticos (+), lógicos (==) y de flujo (<<).



```
    // Sobrecarga de aplicarDescuento
    void aplicarDescuento(double porcentaje){
        precio -= (precio * (porcentaje / 100.0));
    }

    void aplicarDescuento(double monto, bool Fijo){
        if (Fijo && precio >= monto) precio -= monto;
    }

    // Calculo polimorfico base
    virtual double calcularPrecioFinal() const{
        return precio;
    }

    // Sobrecarga de operadores
    double operator+(const Vehiculo& v) const{
        return precio + v.precio;
    }

    bool operator==(const Vehiculo& v) const{
        return (year == v.year && precio == v.precio);
    }

    friend ostream& operator<<(ostream& os, const Vehiculo& v){
        return os << "Marca: " << v.marca << "\nYear: " << v.year << "\nPrecio Base: $" << v.precio;
    }
};
```

Herencia y Polimorfismo: Las clases Automovil y Motocicleta heredan públicamente de Vehiculo y redefinen el método virtual calcularPrecioFinal(). Al recorrer el arreglo de punteros Vehiculo*, la llamada a este método ejecuta dinámicamente el cálculo específico según la clase hija (impuesto del 10% para el automóvil y recargo del 15% para motocicletas de cilindrada ≥ 500 cc).



```
// Clase derivada "Automovil"
class Automovil : public Vehiculo{
private:
    int puertas;

public:
    Automovil() : Vehiculo(), puertas(4){}

    Automovil(const char* m, int y, double p, int numPuertas)
        : Vehiculo(m, y, p), puertas(numPuertas){}

    void setNumPuertas(int p){
        puertas = p;
    }

    int getNumPuertas() const{
        return puertas;
    }

    // 10%
    double calcularPrecioFinal() const{
        return getPrecio() * 1.10;
    }

};

// Clase derivada "Motocicleta"
class Motocicleta : public Vehiculo{
private:
    int cilindrada;

public:
    Motocicleta() : Vehiculo(), cilindrada(125){}

    Motocicleta(const char* m, int y, double p, int cc)
        : Vehiculo(m, y, p), cilindrada(cc){}

    void setCilindrada(int cc){
        cilindrada = cc;
    }

    int getCilindrada() const{
        return cilindrada;
    }

    // Recargo del 15% si la cilindrada >= 500 cc
    double calcularPrecioFinal() const{
        return (cilindrada >= 500) ? getPrecio() * 1.15 : getPrecio();
    }
};
```

Resultado en consola

Para la impresión de los escenarios solicitados, se definieron los siguientes vehículos:

```
159     Automovil auto1("Ferrari", 2026, 6500000.00, 4);
160     cout << auto1 << endl;
161
162     Automovil auto2 = auto1; // Llama al constructor de copia
163     cout << "\nauto2 (copia):\n" << auto2 << endl << endl;
164
165     cout << "ESCENARIO 2\n" << endl;
166     Motocicleta moto1("BMW", 2026, 750000.00, 725);
```

A continuación, se documenta la ejecución por consola del programa demostrando el cumplimiento de los cuatro escenarios de prueba solicitados:

Escenario 1: Construcción y Copia Profunda (Deep Copy)

En este escenario solicitado se comprueba la instanciación de un objeto auto1 de tipo Automovil mediante el constructor parametrizado (reservando memoria dinámica en el Heap para la marca "Ferrari"). Posteriormente, al instanciar auto2 = auto1, se invoca explícitamente el constructor de copia profunda, mostrando el mensaje en consola "Copia hecha y memoria dinamica para 'marca'". Esto garantiza que ambos objetos manejan direcciones de memoria totalmente independientes en el Heap.

```
ESCENARIO 1

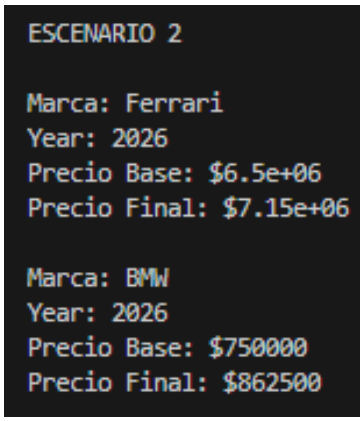
Marca: Ferrari
Year: 2026
Precio Base: $6.5e+06
Copia hecha y memoria dinamica para "marca".


auto2 (copia):
Marca: Ferrari
Year: 2026
Precio Base: $6.5e+06
```

Escenario 2: Cálculo Polimórfico de Precios

Se evalúa el comportamiento polimórfico mediante un arreglo de punteros a la clase base (Vehiculo* inventario[]). Para el objeto de tipo Automovil (auto1), el método calcularPrecioFinal() calcula e incrementa un 10% de impuesto sobre el precio base ($6,500,000.00 a $7,150,000.00).

Para el objeto de tipo Motocicleta (moto1 con cilindrada de 725 cc), el método redefinido detecta que la cilindrada es mayor o igual a 500 cc y aplica automáticamente un recargo del 15% ($750,000.00 a $862,000.00).

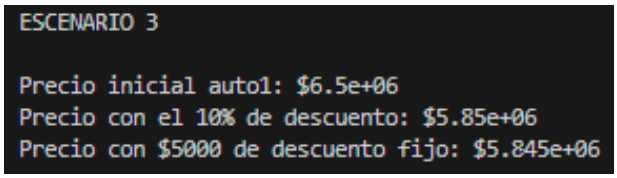


Escenario 3: Aplicación de Descuentos (Sobrecarga de Funciones)

Se prueba la sobrecarga del método aplicarDescuento sobre el objeto auto1:

Descuento Porcentual: Se invoca aplicarDescuento(10.0) reduciendo el precio base un 10% ($6,500,000.00 a $5,850,000.00).

Descuento por Monto Fijo: Se invoca aplicarDescuento(5000.0, true) restando el monto fijo de $5,000 previa verificación de saldo suficiente ($5,850,000.00 a $5,845,000.00).



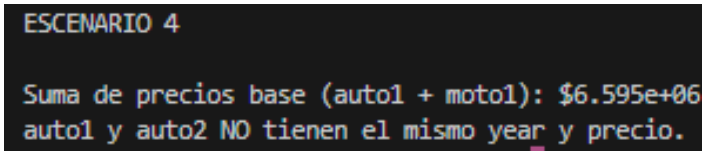
Escenario 4: Evaluación de Operadores Sobrecargados

Verificar el correcto funcionamiento de los operadores sobrecargados:

Operador +: Retorna la suma escalar de los precios base del vehículo auto1 (tras los descuentos) y moto1 ($5,845,000.00 + $750,000.00 = $6,595,000.00).

Operador ==: Compara los atributos year y precio de auto1 y auto2. Evalúa como falso (NO tienen el mismo year y precio) debido a que el precio de auto1 fue modificado previamente en el Escenario 3, mientras que auto2 mantuvo su precio original gracias a la independencia de memoria lograda con la copia profunda.

Operador <<: Utilizado en todos los escenarios como función amiga (friend) para imprimir de forma clara y formateada los datos de cada vehículo.



Conclusiones

La realización de esta práctica integradora permitió resolver de manera efectiva los fallos de estabilidad del sistema anterior de Guanajuato Motors, logrando un núcleo de software sólido, escalable y libre de fugas de memoria (memory leaks). En base a ello, podemos concluir lo siguiente:

Gestión de memoria dinámica y copia profunda (Deep Copy): La aplicación rigurosa de la Regla de los Tres en la clase Vehiculo (constructor parametrizado con new[], constructor de copia profunda y operador de asignación =) demostró ser fundamental cuando se trabaja con datos de tamaño variable en el Heap. Reservar bloques de memoria independientes evita que múltiples objetos compartan punteros al mismo espacio físico, eliminando el riesgo de accesos cruzados no autorizados y la aparición de punteros colgantes (dangling pointers) al destruir objetos.

Encapsulamiento y control de estado: El aislamiento de los datos miembro mediante el modificador de acceso private y su manipulación controlada a través de métodos getters y setters garantizó la integridad del estado vehicular, previniendo asignaciones inconsistentes como precios nulos o negativos.

Sobrecarga como herramienta de versatilidad: La sobrecarga de funciones miembros (aplicarDescuento) otorgó flexibilidad a la interfaz pública de la clase para modelar diferentes reglas de negocio (descuentos porcentuales vs. montos fijos). Por su parte, la sobrecarga de operadores (+, == y <<) permitió realizar operaciones intuitivas directamente sobre los objetos del dominio, mejorando notablemente la legibilidad del código.

Polimorfismo en arquitecturas orientadas a objetos: El uso de una jerarquía de clases con Vehiculo como clase base y las clases derivadas Automovil y Motocicleta demostró la potencia del polimorfismo mediante métodos virtuales. Administrar la flotilla a través de un arreglo de punteros Vehiculo* permitió que cada objeto ejecutara su propia lógica de cálculo de precio final de manera transparente y desacoplada en tiempo de ejecución.

En conclusión, el desarrollo del proyecto reafirma que la combinación de los pilares de la Programación Orientada a Objetos y una administración de recursos en el lenguaje de programación C++ son indispensables para construir software de grado industrial, eficiente y libre de errores de gestión de memoria.